Nexo Solar

DOCUMENTACIÓN DEL PROYECTO | HACKATHON BYECODE | 10 OCTUBRE 2026

Creamos un asistente de seguimiento comercial que ayuda a decidir qué hacer después con cada cotización solar, cuándo hacerlo y con qué evidencia. Conectamos al cliente, al equipo comercial y a los analistas en un mismo flujo.

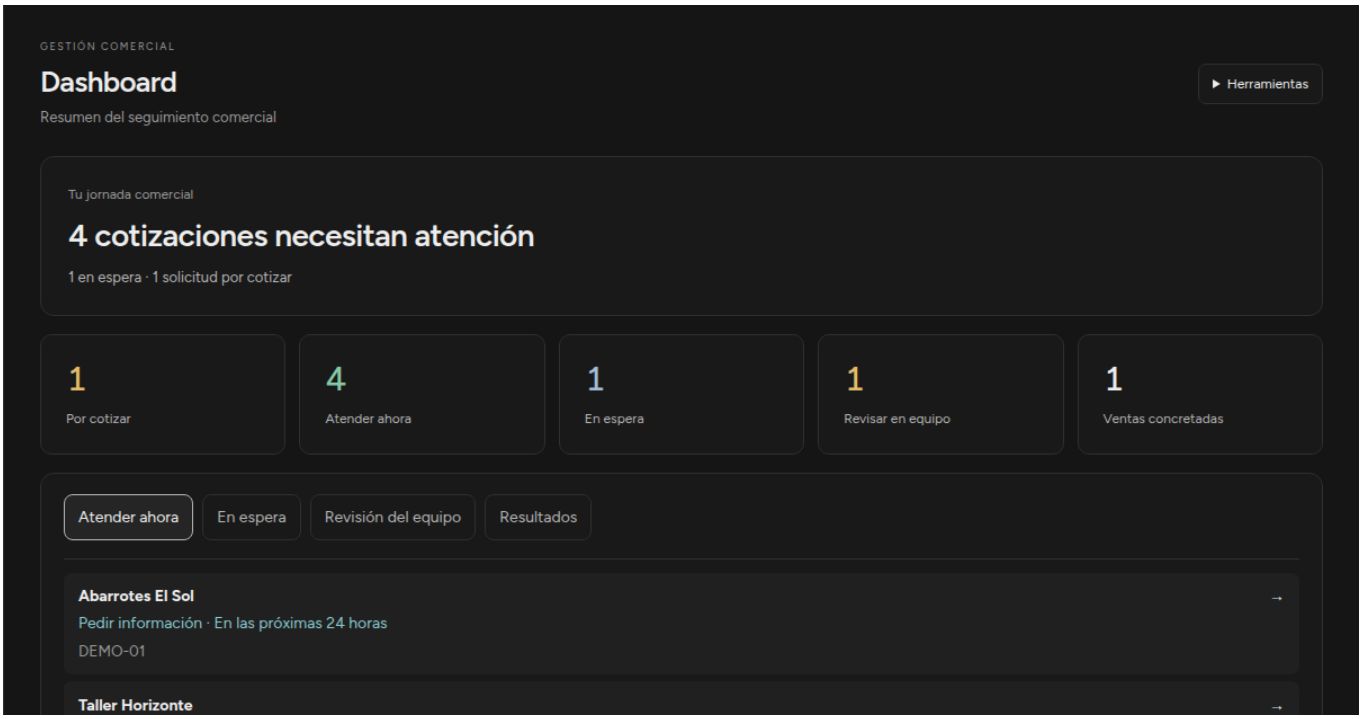
El problema que resolvemos

Una cotización genera aperturas, preguntas y silencios. Esas señales pueden confundirse con intención de compra. Nuestro sistema organiza el historial y recomienda responder, pedir información, esperar, revisar con el equipo o dejar de actuar.

Usuario	Qué puede hacer
Cliente	Solicitar una cotización, revisar su propuesta, hacer preguntas, cambiar preferencias de contacto y aceptar o rechazar.
Agente comercial	Revisar pendientes, publicar propuestas, usar borradores revisados y cerrar ventas con un motivo.
Analista	Consultar consumo, aparatos y datos faltantes antes de confirmar una propuesta.

Estado actual

Tenemos portal, dashboard, persistencia en Tiger Data, integración con Gemini y despliegue en Render. El asistente está configurado en modo borrador: prepara propuestas, pero el equipo revisa antes de enviar respuestas. La integración de correo con n8n está implementada; su entrega en nube debe comprobarse antes de mostrarla como operativa.



Captura del dashboard en una revisión local con casos ficticios. La interfaz ofrece indicadores y detalles desplegables.

Aplicación: https://nexo-solar.onrender.com
Repositorio: https://github.com/luisangelhdzz/nexo-solar

Del formulario al cierre

FLUJO FUNCIONAL

Paso	Lo que hacemos
1. Solicitud	El cliente registra proyecto, contacto, objetivo y preferencias. Puede añadir recibo, consumo y aparatos. No tiene que elegir un kit.
2. Recepción	Creamos el caso y guardamos sus eventos. Antes de publicar una propuesta, el monto puede estar por definir.
3. Revisión	El dashboard muestra el siguiente paso. El analista consulta los datos faltantes y la viabilidad técnica.
4. Propuesta	El equipo revisa una solución interna, su precio y alcance, y publica la propuesta en el portal.
5. Seguimiento	Cada pregunta, vista o restricción se registra. Recalculamos la recomendación y conservamos la decisión anterior.
6. Cierre	El cliente acepta o rechaza, o el equipo registra venta concretada, no concretada o cancelada con su motivo.
7. Aprendizaje	Analizamos factores del historial y generamos lecciones para futuras revisiones. Las hipótesis no sustituyen el motivo declarado.

Análisis de consumo: un ejemplo verificable

En Papelería Punto Claro registramos 900 kWh durante 60 días: el promedio es 15 kWh diarios y 450 kWh por 30 días. Para dos computadoras de 150 W, usadas 8 horas durante 26 días, estimamos 62.4 kWh al mes. Son datos ficticios de demostración.

Fórmula por aparato: cantidad × watts × horas al día × días al mes ÷ 1000. Si falta la potencia del aire acondicionado, no presentamos la suma parcial como consumo total del inventario. Los cuartos describen el inmueble; no permiten dimensionar paneles.

Cómo se usa en el dashboard

Abrimos Cotizaciones, elegimos un caso y desplegamos “Consultar al asistente como analista”. Podemos preguntar: “¿Qué datos de consumo faltan y qué aparatos debemos revisar?”. La consulta es interna y no envía mensajes al cliente.

El dimensionamiento solar, el ahorro y el precio final requieren evaluación técnica. No convertimos directamente pesos del recibo a kWh.

Quién toma las decisiones

IA CON CONTEXTO Y REGLAS VERIFICABLES

Gemini interpreta mensajes y propone resúmenes, borradores y análisis internos. El backend calcula el estado y la acción autorizada mediante reglas determinísticas. Un guardián revisa la recomendación antes de publicarla; al enviar, comprobamos nuevamente las restricciones.

Qué recibe Gemini

Le proporcionamos historial del caso, restricciones, decisión vigente, conocimiento comercial aprobado, ejemplos, lecciones seleccionadas y datos energéticos calculados. Limpiamos nombres, correos y teléfonos del contexto; no prometemos anonimización absoluta de cualquier texto libre. Tratamos los mensajes del cliente como datos, no como instrucciones que cambien las reglas.

Regla	Comportamiento implementado
R1	Caso cerrado: únicamente no actuar.
R2	Si el cliente prohíbe el contacto, bloqueamos seguimiento, solicitudes de información y respuestas comerciales.
R3	Antes de la fecha mínima de contacto, respetamos la espera.
R4	Tras un contacto o una solicitud de información, esperamos 24 horas; un mensaje posterior del cliente permite reevaluar.
R5	Una apertura no confirma compra ni justifica descuentos o urgencia.
R6	Reconocemos información faltante; las restricciones prevalecen. La demostración utiliza datos ficticios.

Autonomía y aprendizaje

Modos disponibles: off, generación manual; draft, borradores y revisiones en segundo plano; auto, envío al portal únicamente si pasa las validaciones. El despliegue actual usa draft. Las preguntas internas nunca activan un envío automático.

Mejoramos el comportamiento con instrucciones, ejemplos, correcciones del equipo y lecciones del historial. No entrenamos un modelo nuevo ni modificamos los pesos de Gemini. Las reglas R1-R6 permanecen en código.

La confianza es baja, media o alta según claridad y evidencia; no es una probabilidad estadística de compra. El riesgo aumenta con ambigüedad, objeciones, manipulación, montos altos o conflictos entre una petición y las restricciones.

Arquitectura y datos

COMPONENTES DEL PROTOTIPO

Portal y dashboard (React + Vite) → API (Node.js + Express) → motor de eventos, reglas y guardián → PostgreSQL en Tiger Data. El backend consulta Gemini para interpretar y redactar. n8n integra la entrega de correos. Render sirve frontend y backend desde un mismo servicio.

Componente	Función
React + Vite	Portal del cliente, dashboard, clientes, cotizaciones y aprendizaje.
Node.js + Express	API, autenticación de consola, eventos, cálculos y flujo comercial.
Google Gemini	Interpretación de lenguaje, borradores y consultas de analistas.
Tiger Data / PostgreSQL	Clientes, eventos, decisiones, historial, lecciones y registros del asistente. No afirmamos que usamos agregados temporales o compresión especial.
n8n	Recibe webhooks y entrega acuse, propuesta y comprobante mediante Gmail o SMTP, si está configurado. No decide el siguiente paso.
Render	Aloja la aplicación; el repositorio incluye la configuración de despliegue.

Modelo de información

Cada evento conserva event_id, case_id, event_type, occurred_at y data. Agrupamos por caso y deduplicamos por event_id. Guardamos decisiones con su versión anterior, interpretaciones, clientes, comprobantes, notificaciones, configuraciones, lecciones, ejecuciones del asistente y correcciones. Los datos de consumo se guardan dentro de la solicitud del caso.

Acceso y privacidad

La consola requiere contraseña al desplegar. Los enlaces del cliente utilizan una credencial aleatoria con vigencia de 30 días; el enlace de una cotización limita el acceso a ese caso. Guardamos el token como hash y lo conservamos en la sesión de la pestaña del navegador. No mostramos un directorio público de empresas.

Las claves y la conexión a la base se guardan en backend/.env localmente y en variables privadas de Render. El .env no se publica. PostgreSQL utiliza SSL con verificación del certificado. Los enlaces de cotización deben compartirse solo con su destinatario.

El portal usa acceso mediante enlaces, no cuentas verificadas por identidad. El comprobante es simulado y no es CFDI. La aplicación no procesa pagos reales.

Verificación y entrega del reto

WHAT SHOULD HAPPEN NEXT?

Requisito del reto	Cómo lo cubrimos
Carga JSONL y errores	Interfaz, API y CLI; validamos líneas y continuamos con las válidas.
Historial y duplicados	Eventos por case_id, deduplicación por event_id y persistencia intercambiable.
IA e incertidumbre	Interpretación con hechos, inferencias, datos faltantes y respaldo si falla el proveedor.
Decisión estructurada	Acción, momento, evidencia, confianza, riesgo y revisión humana; validación determinística.
Actualización	Nuevos eventos recalculan el caso; conservamos la decisión anterior.
Interfaz y exportación	Lista, detalle, historial y JSON de recomendaciones.
Reproducibilidad	Hora de manifest.json para el kit; pruebas locales y comandos de instalación.

Resultados del kit público

Caso	Resultado después de events.jsonl y updates.jsonl
P01	Aclarar duda, inmediato.
P02	Esperar hasta una fecha; una apertura sola no confirma compra.
P03	Esperar hasta la fecha pedida por el cliente.
P04	Revisión humana; contacto prohibido.
P05	No actuar; caso cerrado.
P06	Antes: esperar por R4. Después del mensaje nuevo: aclarar duda.

Verificación del 10 de octubre de 2026: 151 pruebas, 149 correctas, 0 fallos y 2 omitidas porque requieren una base PostgreSQL de pruebas dedicada. Comprobamos además compilación, formulario en escritorio/móvil y una respuesta interna real de Gemini. No ejecutamos las pruebas destructivas contra la base compartida.

El reto asigna 30 puntos a decisiones, 25 a reglas e inacción, 20 a contexto y actualización, 15 a evidencia e incertidumbre y 10 a demostración y reproducibilidad. El video de 3 minutos queda dentro del máximo de 5 minutos del documento.

Fuente: Reto_Byecode_Participantes_24h (1).pdf, secciones 2, 4-8. La carga de datos del kit para grabar debe hacerse en un entorno local de prueba, conservando la base compartida.

Ejecución y presentación

GUÍA PARA EL EQUIPO

Cómo ejecutar el proyecto

Requisito: Node.js 18.17 o superior. Desde la raíz: npm run setup instala dependencias y compila; npm start sirve la aplicación en el puerto 3001. Para desarrollar, ejecutamos npm run dev:backend y npm run dev:frontend en terminales separadas.

Copiamos backend/.env.example a backend/.env y configuramos DATABASE_URL, SSL y su certificado, ADMIN_PASSWORD, AI_PROVIDER=gemini y GEMINI_API_KEY. El modelo se configura con GEMINI_MODEL. ASSISTANT_AUTOPILOT=draft mantiene revisión del equipo. Las variables N8N_WEBHOOK_URL y N8N_WEBHOOK_SECRET habilitan la integración de correo.

Comprobaciones: npm run db:init --prefix backend verifica tablas; npm run ai:check verifica Gemini; npm test ejecuta pruebas. npm run evaluate:kit exporta recomendaciones usando almacenamiento en memoria, separado de la base compartida. No hace falta reiniciar la base para grabar.

Video de 3 minutos: orden recomendado

Tiempo	Qué mostramos
00:00-00:20	Problema y propuesta de Nexo Solar.
00:20-00:45	Solicitud desde el portal y datos opcionales de consumo.
00:45-01:05	Dashboard: pendientes, espera, revisión y acceso a una cotización.
01:05-01:45	Asistente y análisis de kWh, aparatos y datos faltantes.
01:45-02:10	Decisión anterior, mensaje nuevo y cambio de recomendación con evidencia.
02:10-02:30	Contacto bloqueado y espera justificada.
02:30-02:50	Cierre de venta, motivo y aprendizaje.
02:50-03:00	Arquitectura, exportación y enlace del proyecto.

Qué preparar antes de grabar

Usamos solo datos ficticios. Antes de grabar, preparamos capturas de portal, dashboard, asistente, historial, cierre y aprendizaje, además de P06 antes/después y P04 del kit. Grabamos cada escena por separado, recortamos esperas y mostramos la evidencia legible. No exponemos contraseñas, tokens, correos personales ni el .env.

Entregables: repositorio con README, nota técnica de máximo una página, JSON de recomendaciones del kit y demostración. Para Devpost faltan los datos específicos del evento, integrantes y enlace final del video. Esta documentación no implica que la entrega ya se haya enviado.